toXo

v2.0

Приложение для управления задачами
и личными финансами

Техническая документация

Полное руководство по архитектуре, функциям и использованию

Автор: TheOverforge | github.com/TheOverforge

Стек технологий

Electron 31 **React 18** **TypeScript 5**

Vite 5 Zustand 4 better-sqlite3 9

TipTap · Recharts · Radix UI · Tailwind CSS · i18next

19 апреля 2026 г. | Windows 10/11

Содержание

1 Обзор приложения **2**
1.1 Ключевые возможности 2

2 Используемые технологии **3**
2.1 Electron 31 3
2.2 React 18 + TypeScript 5 4
2.3 Vite 5 4
2.4 Zustand 4 5
2.5 better-sqlite3 9 5
2.6 TipTap 2 5
2.7 Recharts 2 5
2.8 Дополнительные зависимости 6

3 Архитектура **6**
3.1 Структура монорепозитория 6
3.2 Поток данных 7
3.3 Главный процесс (main) 8
3.3.1 index.ts — жизненный цикл 8
3.3.2 windowManager.ts — окно и визуальный эффект 8
3.3.3 notificationScheduler.ts — уведомления 8
3.4 Preload 8
3.5 Рендерер 9

4 База данных **9**
4.1 Путь к файлу 9
4.2 Схема таблиц 9
4.2.1 tasks 9
4.2.2 subtasks 10
4.2.3 categories 10
4.2.4 Финансовые таблицы 10
4.3 Класс AppDatabase 11

5 Страницы приложения **11**
5.1 Задачи (Tasks) 11
5.1.1 Мультиселект 12
5.1.2 Переименование inline (F2) 12
5.1.3 Контекстное меню задачи 12
5.1.4 Undo/Redo 12
5.2 Аналитика (Analytics) 12
5.3 Календарь (Calendar) 13

5.4 Финансы (Finance) 13
5.5 Настройки (Settings) 14

6 IPC API 14
6.1 tasks 14
6.2 subtasks 15
6.3 categories / analytics / finance / settings 15
6.4 Push-события (main → renderer) 16

7 Горячие клавиши 16

8 Сборка и дистрибуция 17
8.1 Скрипты 17
8.2 electron-builder 17
8.3 Первая установка на новой машине 17

9 Настройки (settings.json) 18

1 Обзор приложения

toXo --- настольное приложение для Windows для управления задачами и личными финансами. Начиная с версии 2.0 приложение полностью переписано: Python + PyQt6 заменён стеком **Electron 31 + React 18 + TypeScript**, при этом формат базы данных SQLite полностью сохранён --- существующие данные из предыдущих версий загружаются без миграции. Интерфейс поддерживает три темы: **Dark**, **Light** и **Glass** (нативный acrylic-blur DWM). Акцентный цвет --- синий `#0a84ff`; документация оформлена в фирменной фиолетовой палитре автора.

1.1 Ключевые возможности

- **Полный CRUD задач** --- создание, просмотр, редактирование, удаление
- **Rich-text редактор** --- TipTap: жирный / курсив / зачёркнутый, заголовки H1--H3, списки, цитаты, блоки кода, произвольный цвет и шрифт
- **Приоритеты** --- три уровня (1 низкий, 2 средний, 3 высокий) + цветовая индикация
- **Закрепление** --- задача фиксируется вверху списка
- **Дедлайны** --- уведомление накануне и в день дедлайна
- **Напоминания** --- произвольное время, системное OS-уведомление
- **Повторения** --- ежедневно / еженедельно / ежемесячно
- **Теги** --- произвольные метки через запятую
- **Подзадачи** --- список чекбоксов с drag-and-drop реордерингом
- **Категории** --- цветные папки; drag-and-drop задач и папок
- **Фильтрация** --- по статусу (все / активные / выполненные / сегодня / архив), категории и поисковому запросу
- **Сортировка** --- по дате (новые / старые), алфавиту, «невыполненные первыми», вручную
- **Архив** --- ручная и авто-архивация выполненных задач
- **Отмена/повтор** --- полная история через Ctrl+Z / Ctrl+Y
- **Дублирование** --- Ctrl+D копирует задачу со всеми полями
- **Мультиселект** --- Ctrl+клик добавляет задачу в выборку, Shift+клик выбирает диапазон, Ctrl+A --- все задачи; массовые операции через BulkActionBar

- **Переименование inline** --- F2 или двойной клик по названию в списке переводит карточку в режим редактирования прямо на месте
- **Страница Аналитика** --- 6 графиков Recharts, KPI-карточки, стрик, периоды 7/30/90 дней и **произвольный диапазон дат**
- **Страница Календарь** --- сетка месяца с дедлайнами, панель дня, быстрое создание/редактирование, **DnD задач в ячейки** (drag-to-navigate через сайдбар), кнопка перехода к задаче (↗)
- **Страница Финансы** --- счета, транзакции (доход/расход/**перевод между счетами**), категории, бюджеты, цели накоплений, 5 графиков
- **Командная палитра** --- Ctrl+K, поиск и запуск любой команды
- **Splash-экран** --- анимированный экран загрузки при запуске
- **Первый запуск** --- автоматическое открытие приветственной задачи на языке приложения
- **Интерактивный туториал** --- 11 шагов с автоматической навигацией по страницам, spotlight на каждом элементе, анимированный переход
- **Локализация** --- русский и английский, переключение без перезапуска
- **Системный трей** --- фоновая работа, быстрые действия из трея
- **Три темы** --- Dark, Light, Glass (acrylic blur)
- **Звуковые уведомления** --- настраиваемый звук напоминаний
- **Экспорт/импорт** --- JSON и CSV
- **Демо-данные** --- предзаполненный набор задач и транзакций (с учётом языка)

2 Используемые технологии

2.1 Electron 31

Оболочка приложения. Electron предоставляет нативный процесс (main) и браузерный контекст (renderer). Ключевые возможности Electron, задействованные в toXo:

API	Использование
`BrowserWindow`	Главное окно с `transparent: true` и `frame: false`

API	Использование
`ipcMain.handle`	Регистрация обработчиков запросов от рендерера
`contextBridge`	Безопасная передача API в рендерер через preload
`Notification`	Системные OS-уведомления (напоминания, дедлайны)
`Tray`	Иконка в системном трее с контекстным меню
`app.requestSingleInstanceLock`	Единственный экземпляр приложения
`SetWindowCompositionAttribute`	Нативный acrylic-blur DWM через Win32 FFI

Важно: transparent + acrylic

Окно создаётся с `transparent: true` и `show: false`. Acrylic применяется через `SetWindowCompositionAttribute` *до* вызова `win.show()` --- это гарантирует корректную работу DWM с первого кадра без серого мигания.

2.2 React 18 + TypeScript 5

Рендерер написан на React 18 с полной типизацией TypeScript. Архитектура UI:

- **Pages** --- `TasksPage`, `AnalyticsPage`, `CalendarPage`, `FinancePage`, `SettingsPage`
- **Widgets** --- `AppSidebar`, `TaskEditor`, `TaskList`, `CommandPalette`, `WindowControls`, `TourOverlay`, `LaunchOverlay`
- **Features** --- `CategoryBar`, `Topbar`, компоненты фильтров
- **Shared UI** --- `Button`, `Dialog`, `DateTimePicker`, `ColorPicker` (на базе Radix UI)

2.3 Vite 5

Сборщик рендерера. Использует `@vitejs/plugin-react` (SWC). Конфигурация:

- Алиас `@/` → `renderer/src/`
- Режим разработки: hot module replacement на `localhost:5173`
- Продакшн: одна связка `dist/assets/index-*.js` + CSS

2.4 Zustand 4

Менеджер состояния. Три основных store:

Store	Содержимое
`useTaskStore`	tasks, subtasks, selectedTaskId, **selectedTaskIds**, filterMode, sortMode, searchQuery, currentCategoryId; методы: toggleSelectTask, selectAll, addToSelection, clearMultiSelect
`useCategoryStore`	categories
`useHistoryStore`	стек undo/redo --- пары `{undo, redo}`

2.5 better-sqlite3 9

Синхронный SQLite в главном процессе Electron. Файл БД: `%APPDATA%\toxo\task` Особенности использования:

- `PRAGMA journal_mode = WAL` --- параллельное чтение без блокировок
- `PRAGMA foreign_keys = ON` --- каскадные удаления подзадач и транзакций
- Синхронный API не требует `async/await` в слое репозитория
- Нативный `.node` модуль требует `asarUnpack` и `electron-rebuild`

2.6 TipTap 2

Rich-text редактор задачи. Расширения:

- **StarterKit** --- базовое форматирование, заголовки, списки, цитаты
- **TextStyle + Color** --- произвольный цвет текста (palette + favorites)
- **FontFamily** --- пользовательские шрифты (системные + добавленные)
- **Underline** --- подчёркивание
- Содержимое хранится как HTML-строка в столбце `description` таблицы `tasks`

2.7 Recharts 2

Библиотека графиков React. Используется в разделах Аналитика и Финансы:

Компонент	Применение
`AreaChart`	Выполненные задачи по дням, накопленный баланс
`BarChart`	Создано vs выполнено, активность по дням недели
`PieChart / RadialBarChart`	Распределение статусов и приоритетов
`LineChart`	Ежедневный денежный поток

2.8 Дополнительные зависимости

Пакет	Назначение
`@radix-ui/*`	Headless примитивы: Dialog, Select, Popover, Dropdown, Checkbox
`tailwindcss 3`	CSS-утилиты, потребляет CSS custom properties тем
`date-fns 3`	Вычисления дат (календарь, аналитика, дедлайны)
`i18next + react-i18next`	Локализация RU/EN, живое переключение языка
`react-dnd 16`	Drag-and-drop: реордеринг задач и подзадач
`electron-builder 24`	Сборка NSIS-установщика для Windows
`concurrently`	Параллельный запуск Vite + Electron в dev-режиме

3 Архитектура

3.1 Структура монорепозитория

Проект организован как npm-воркспейс с тремя пакетами:

```
toXo/
|-- package.json            (workspaces: main, preload, renderer)
|-- electron-builder.json5
|-- tsconfig.base.json
|-- main/
|   `-- src/
|       |-- index.ts         (app lifecycle, BrowserWindow)
|       |-- windowManager.ts (DWM titlebar, acrylic)
|       |-- tray.ts
|       |-- singleInstance.ts
|       |-- notificationScheduler.ts
|       |-- demoData.ts
|       |-- ipc/
```

```
|       |   |-- tasks.ipc.ts
|       |   |-- categories.ipc.ts
|       |   |-- analytics.ipc.ts
|       |   |-- finance.ipc.ts
|       |   `-- settings.ipc.ts
|       `-- db/
|           |-- database.ts
|           |-- migrations.ts
|           |-- taskRepo.ts
|           |-- categoryRepo.ts
|           |-- analyticsRepo.ts
|           `-- financeRepo.ts
|-- preload/
|   `-- src/preload.ts       (contextBridge → window.electronAPI)
`-- renderer/
    `-- src/
        |-- main.tsx
        |-- app/             (App.tsx, ThemeProvider, i18nProvider)
        |-- pages/           (tasks/, analytics/, calendar/, finance
           /, settings/)
        |-- widgets/         (AppSidebar, TaskEditor, CommandPalette,
            ...)
        |-- features/        (CategoryBar, Topbar, filter-tasks, ...)
        |-- entities/        (task/store.ts, category/store.ts)
        `-- shared/
            |-- api/ipc.ts типизированные  ( обёртки window.
               electronAPI)
            |-- i18n/        (ru.ts, en.ts)
            |-- lib/         (undoRedo.ts, dateUtils.ts, sounds.ts)
            `-- ui/          (Button, Dialog, DateTimePicker,
               ColorPicker)
```

3.2 Поток данных

Принцип IPC

Рендерер **никогда** не импортирует `electron` напрямую. Весь доступ к нативным функциям происходит через `window.electronAPI`, которое экспозирует preload-скрипт через `contextBridge`. Каждый вызов в рендерере оборачивается типизированной функцией в `shared/api/ipc.ts`.

Схема вызова данных:

```
Renderer (React)
  → ipc.tasks.list()          // shared/api/ipc.ts
  → window.electronAPI.tasks.list()
  → ipcRenderer.invoke('tasks:list')
  → ipcMain.handle('tasks:list')   // main/ipc/tasks.ipc.ts
```

```
  → db.tasks.listTasks()            // main/db/taskRepo.ts
  → better-sqlite3 .prepare().all()
```

3.3 Главный процесс (main)

3.3.1 index.ts — жизненный цикл

- `app.whenReady()` → инициализация БД → регистрация IPC-хендлеров → создание окна и трея
- `app.requestSingleInstanceLock()` → второй запуск фокусирует существующее окно
- `app.on('before-quit')` → корректное завершение планировщика уведомлений

3.3.2 windowManager.ts — окно и визуальный эффект

Создаёт `BrowserWindow` с параметрами `transparent: true`, `frame: false`, `show: false`. Последовательность запуска:

1. `ready-to-show`: вызов `applyAcrylic(win)` → `SetWindowCompositionAttribute` (устанавливает acrylic на скрытом окне)
2. `win.show()` → DWM читает атрибуты HWND и отрисовывает acrylic с первого кадра
3. При максимизации и восстановлении: повторный вызов `applyAcrylic`

3.3.3 notificationScheduler.ts — уведомления

`setInterval(60_000)` опрашивает БД:

- `taskRepo.getDueReminders()` → отправляет `new Notification(...).show()`, ставит `remind_shown=1`
- `taskRepo.getOverdueDeadlines()` → уведомление о просроченном дедлайне
- Pushes IPC-events `notification:reminderFired` / `notification:deadlineFired` в рендерер (для воспроизведения звука)

3.4 Preload

Файл `preload/src/preload.ts` использует `contextBridge.exposeInMainWorld('` где `api` --- объект с типизированными оболочками `ipcRenderer.invoke` для всех каналов. Полный тип поверхности экспортируется как `ElectronAPI` и переиспользуется в рендерере.

3.5 Рендерер

Точка входа: `renderer/src/main.tsx`. Корневой компонент: `App.tsx`. Инициализация при запуске (`useEffect`):

1. `fetchTasks()` + `fetchCategories()` параллельно
2. Проверка `tour_suppressed` → отложенный показ туториала
3. Минимальная задержка 4 с (время показа splash-экрана)
4. Проверка `welcome_shown` → `ensureWelcomeTask()` → `selectTask(id)` при первом запуске (задача создаётся на языке приложения через `readLang()`)
5. `setAppReady(true)` → `LaunchOverlay` начинает fade-out (60 мс → setFading, 480 мс → unmount)

4 База данных

4.1 Путь к файлу

- **Windows:** `%APPDATA%\toxo\tasks.sqlite`
- **Настройки:** `%APPDATA%\toxo\settings.json`

Совместимость с v1.x

Схема базы данных полностью идентична схеме версии 1.x (Python/PyQt6). Существующий файл `tasks.sqlite` открывается новой версией без каких-либо изменений.

4.2 Схема таблиц

4.2.1 tasks

Столбец	Тип	Описание
`id`	INTEGER PK	Автоинкремент
`title`	TEXT	Заголовок задачи
`description`	TEXT	HTML-содержимое TipTap редактора
`is_done`	INTEGER	0/1
`priority`	INTEGER	0 нет, 1 низкий, 2 средний, 3 высокий
`is_pinned`	INTEGER	0/1, закреплённая задача
`category_id`	INTEGER FK	Ссылка на `categories.id`
`tags`	TEXT	Теги через запятую

Столбец	Тип	Описание
recurrence	TEXT	daily / weekly / monthly / NULL
reminder_at	TEXT	ISO 8601, время напоминания
deadline_at	TEXT	ISO 8601, дедлайн
remind_shown	INTEGER	0/1, уведомление уже показано
is_archived	INTEGER	0/1
created_at	TEXT	ISO 8601
completed_at	TEXT	ISO 8601 / NULL

4.2.2 subtasks

Столбец	Тип	Описание
id	INTEGER PK	Автоинкремент
task_id	INTEGER FK	Ссылка на tasks.id (CASCADE DELETE)
title	TEXT	Текст подзадачи
is_done	INTEGER	0/1
position	INTEGER	Порядок в списке

4.2.3 categories

Столбец	Тип	Описание
id	INTEGER PK	Автоинкремент
name	TEXT	Название категории
color	TEXT	HEX-цвет, например #0a84ff
position	INTEGER	Порядок в списке

4.2.4 Финансовые таблицы

Таблица	Содержимое
fin_accounts	Счета: name, type (checking/savings/cash/credit/investment), balance, currency, color
fin_categories	Категории транзакций: name, type (income/expense), color, icon
fin_transactions	Транзакции: type (income/expense/transfer), amount, currency, category_id, account_id, **to_account_id** (для переводов), date, title, note, tags, is_recurring
fin_budgets	Бюджеты: category_id, limit_amount, period (monthly/weekly)

Таблица	Содержимое
fin_goals	Цели накоплений: name, target_amount, current_amount, currency, color, deadline

4.3 Класс AppDatabase

main/src/db/database.ts компонует четыре репозитория:

```
export class AppDatabase {
  readonly db:       Database.Database;
  readonly tasks:    TaskRepo;
  readonly categories: CategoryRepo;
  readonly analytics:  AnalyticsRepo;
  readonly finance:    FinanceRepo;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    runMigrations(this.db);      // migrations.ts v0..v13
    this.db.pragma('journal_mode = WAL');
    this.db.pragma('foreign_keys = ON');
    this.tasks      = new TaskRepo(this.db);
    this.categories = new CategoryRepo(this.db);
    this.analytics  = new AnalyticsRepo(this.db);
    this.finance    = new FinanceRepo(this.db);
  }
}
```

5 Страницы приложения

5.1 Задачи (Tasks)

Главная страница. Разделена на три колонки:

1. **CategoryBar** --- горизонтальная полоса цветных категорий + кнопка «+»
2. **TaskList** --- список задач с фильтрацией, сортировкой, поиском, drag-and-drop
3. **TaskEditor** --- правая панель: заголовок, TipTap, приоритет, закреп, дедлайн, напоминание, повторение, теги, подзадачи, кнопки действий

Topbar над списком содержит кнопки фильтра статуса (Все / Активные / Выполненные / Сегодня / Архив), поисковое поле и выпадающий список сортировки.

5.1.1 Мультиселект

- **Ctrl+клик** --- добавить/убрать задачу из выборки
- **Shift+клик** --- выбрать диапазон от последнего кликнутого (anchor) до текущего
- **Ctrl+Shift+клик** --- добавить диапазон к существующей выборке
- **Ctrl+A** --- выбрать все видимые задачи (использует `e.code = 'KeyA'`, не зависит от раскладки клавиатуры)

Состояние хранится в `useTaskStore`: `selectedTaskIds: number[]`, `lastSelectedIdRef` (anchor) живёт в компоненте как `useRef`.

5.1.2 Переименование inline (F2)

Нажатие `F2` или двойной клик по названию задачи в `TaskItem` переключает в режим редактирования: заголовок заменяется `<input>` с нижним подчёркиванием в цвет акцента. `Enter` --- сохранить, `Escape` --- отмена, `blur` --- автосохранение. Использует `useTaskStore.updateFields`.

5.1.3 Контекстное меню задачи

Правая кнопка мыши или кнопка «⋯» на карточке открывает меню:

- Открыть / Дублировать / Удалить
- Закрепить / Открепить
- Установить приоритет 1 / 2 / 3
- В архив / Из архива

5.1.4 Undo/Redo

`useHistoryStore` хранит стек пар `{undo, redo}`. Каждая деструктивная операция (удаление, архивация) кладёт в стек снапшот задачи и вызов `tasks:restore` для восстановления. Ctrl+Z и Ctrl+Y выполняют соответствующие функции.

5.2 Аналитика (Analytics)

Шесть графиков Recharts + четыре KPI-карточки. Период выбирается кнопками: **7 дней / 30 дней / 90 дней / Произвольный**. При выборе «Произвольного» диапазона появляются поля ввода дат «с» и «по»; данные перезапрашиваются после подтверждения.

График	Описание
Completed per Day	AreaChart: выполненных задач в день
Created vs Completed	BarChart: создано vs выполнено за период
Status Distribution	PieChart: активные / выполненные / в архиве

График	Описание
By Weekday	BarChart: средняя продуктивность по дням недели
By Category	BarChart: выполнено задач по категориям
Priority Distribution	RadialBarChart: распределение приоритетов

KPI-карточки: **Streak** (серия дней с выполненными задачами), **Done Total**, **Completion Rate**, **Best Weekday**.

5.3 Календарь (Calendar)

Сетка месяца и боковая панель дня:

- Каждая ячейка показывает до 2 чипов задач с дедлайном на этот день
- Клик по ячейке → загрузка задач в панели дня справа
- **Быстрое создание** --- поле ввода в нижней части панели; `Enter` или кнопка «+»
- **Быстрое редактирование** --- клик по чипу в сетке *или* по строке в панели → режим редактирования с полем ввода, кнопками Сохранить / Отмена (`Enter` / `Escape`)
- **Перетаскивание (DnD)** --- задача из `TaskList` тащится на ячейку календаря → дедлайн устанавливается на 12:00 этого дня. Поскольку страницы на разных вкладках: при наведении на кнопку **«Календарь»** в сайдбаре на 600 мс приложение автоматически переключает вкладку (drag-to-navigate)
- **Кнопка ↗ «Открыть в задачах»** --- у каждой задачи в панели дня; при клике переключает на вкладку Tasks и выделяет задачу в редакторе
- Кнопка «Сегодня» возвращает к текущей дате

5.4 Финансы (Finance)

Пять вкладок:

1. **Обзор** --- суммарный баланс по счетам, графики потоков и расходов по категориям
2. **Транзакции** --- таблица с фильтром по периоду и типу, добавление/удаление
3. **Счета** --- карточки счетов с балансом; добавление через диалог

4. **Бюджеты** --- прогресс-бар расходов относительно лимита за текущий период
5. **Цели** --- карточки целей накоплений с прогрессом; кнопка «Пополнить»

Переводы между счетами. Транзакция типа `transfer` снимает сумму с одного счёта и зачисляет на другой. В Quick-add появляется третья кнопка «Перевод»; при выборе отображается дополнительный селектор «Счёт назначения». Реализовано атомарно через SQLite-транзакцию (`FinanceRepo.addTransfer`). Схема БД расширена в миграции v13: `ALTER TABLE fin_transactions ADD COLUMN to_account_id INTEGER`.

При вызове `clearAll` или `loadDemo` из настроек страница получает custom event `finance:reload` и перезагружает все данные.

5.5 Настройки (Settings)

Секция	Параметры
Внешний вид	Тема (Dark / Light / Glass), язык (RU / EN)
Уведомления	Включить/выключить, выбор звука (6 пресетов + тест)
Данные	Загрузить демо-данные, Очистить всё, Экспорт JSON/CSV, Импорт JSON
О приложении	Версия, ссылка на репозиторий, кнопка «Начать тур»

6 IPC API

Все каналы зарегистрированы через `ipcMain.handle`. Рендерер обращается к ним через `window.electronAPI` (preload).

6.1 tasks

Канал	Описание
`tasks:list`	Список всех задач
`tasks:get`	Задача по id
`tasks:create`	Создать задачу `{title?, description?, categoryId?}`
`tasks:update`	Обновить title + description
`tasks:updateFields`	Обновить произвольные поля
`tasks:delete`	Удалить задачу
`tasks:setDone`	Установить is_done
`tasks:setPriority`	Установить priority (0--3)

Канал	Описание
`tasks:setPinned`	Закрепить/открепить
`tasks:setRecurrence`	Установить повторение
`tasks:setReminder`	Установить напоминание (ISO)
`tasks:setDeadline`	Установить дедлайн (ISO)
`tasks:setCategory`	Привязать к категории
`tasks:setTags`	Установить теги (строка)
`tasks:reorder`	Перепорядочить задачи (массив id)
`tasks:archive`	Архивировать/разархивировать
`tasks:archiveCompleted`	Архивировать все выполненные старше даты
`tasks:clearArchive`	Удалить все архивные
`tasks:duplicate`	Дублировать задачу
`tasks:restore`	Восстановить из снапшота (undo)
`tasks:getByDeadlineDate`	Задачи с дедлайном на дату (YYYY-MM-DD)
`tasks:logEvent`	Логировать событие в аналитику
`tasks:exportJson`	Экспорт в JSON-файл
`tasks:exportCsv`	Экспорт в CSV-файл
`tasks:importJson`	Импорт из JSON-файла
`tasks:loadDemo`	Загрузить демо-данные (с учётом языка)
`tasks:clearAll`	Удалить все данные (задачи + финансы)
`tasks:ensureWelcomeTask`	Найти или создать приветственную задачу

6.2 subtasks

Канал	Описание
`subtasks:list`	Подзадачи по taskId
`subtasks:add`	Добавить подзадачу
`subtasks:setDone`	Установить is_done
`subtasks:rename`	Переименовать
`subtasks:delete`	Удалить
`subtasks:reorder`	Перепорядочить
`subtasks:counts`	Счётчики all/done по всем задачам

6.3 categories / analytics / finance / settings

Группа	Каналы
`categories:*`	list, add, update, delete, reorder

Группа	Каналы
`analytics:*`	getKpi, completedPerDay, createdVsCompleted, statusDistribution, completedByWeekday, completedByCategory, priorityDistribution
`fin:accounts:*`	list, add, update, delete
`fin:categories:*`	list, add, update, delete
`fin:tx:*`	list, add, **transfer**, update, delete, summary, dailyFlow, expenseByCategory, cumulativeBalance, totalBalance
`fin:budgets:*`	list, add, update, delete, getSpent
`fin:goals:*`	list, add, update, delete, addTo
`settings:*`	get, set, getAll
`shell:*`	showSaveDialog, showOpenDialog

6.4 Push-события (main → renderer)

Канал	Описание
`notification:reminderFired`	Сработало напоминание → воспроизвести звук
`notification:deadlineFired`	Сработал дедлайн → воспроизвести звук
`window:maximized`	Окно максимизировано/восстановлено (boolean)
`tray:newTask`	Нажато «Новая задача» в меню трея

7 Горячие клавиши

Сочетание	Действие
`Ctrl+N`	Новая задача
`Ctrl+K`	Открыть / закрыть командную палитру
`Ctrl+Z`	Отменить последнее действие
`Ctrl+Shift+Z` / `Ctrl+Y`	Повторить отменённое действие
`Ctrl+D`	Дублировать выбранную задачу
`Ctrl+P`	Закрепить / открепить задачу
`Ctrl+1`	Установить приоритет «Низкий»
`Ctrl+2`	Установить приоритет «Средний»
`Ctrl+3`	Установить приоритет «Высокий»

Сочетание	Действие
`Ctrl+A`	Выбрать все видимые задачи (мультиселект)
`Ctrl+клик`	Добавить / убрать задачу из выборки
`Shift+клик`	Выбрать диапазон от anchor до текущей задачи
`F2`	Переименовать выделенную задачу inline
`Delete`	Удалить выбранную задачу
`Escape`	Снять выделение / закрыть диалог / отмена переименования
`Enter`	Подтвердить ввод (быстрое создание / переименование / редактирование)

8 Сборка и дистрибуция

8.1 Скрипты

Команда	Описание
`npm run dev:app`	Dev-режим: Vite dev server + Electron с hot reload
`npm run build`	Полная продакшн-сборка: renderer → preload → main
`npm run package`	`build` + `electron-builder` → NSIS .exe установщик
`npm run rebuild-native`	Пересборка `better-sqlite3` под текущий Electron

8.2 electron-builder

Конфигурация `electron-builder.json5`:

- Цель: **NSIS** (Windows x64)
- `asarUnpack`: `node_modules/better-sqlite3/**/*` --- нативный `.node` файл должен быть вне asar-архива
- Иконка: `build/app_icon.ico`
- AppId: `com.theoverforge.toxo`

8.3 Первая установка на новой машине

```
git clone https://github.com/TheOverforge/toxo
cd toxo
npm install
npm run rebuild-native     # пересборка better-sqlite3
npm run dev:app            # режимразработки
# или
npm run package            # создатьустановщик
```

Требования к системе

- Windows 10 версии 1903+ или Windows 11
- Node.js ≥ 18 LTS
- Python 3 (только для сборки нативных модулей через `node-gyp`)
- Visual C++ Build Tools

9 Настройки (settings.json)

Файл `%APPDATA%\toxo\settings.json` хранит пользовательские настройки в JSON:

Ключ	Тип	Описание
`theme`	string	`"dark"` / `"light"` / `"glass"`
`language`	string	`"ru"` / `"en"`
`notification_sound`	string	Ключ звука напоминания
`notifications_enabled`	boolean	Включены ли уведомления
`welcome_shown`	boolean	Приветственная задача уже показана
`tour_suppressed`	boolean	Туториал отключён
`color_recent`	string[]	Недавно использованные цвета
`color_favorites`	string[]	Избранные цвета
`custom_fonts`	string[]	Добавленные пользователем шрифты